

NEXT SDK Documentation

The NEXT SDK allows you to integrate your dApps with the Bitkub NEXT Wallet, which provides native support for the KUB network. It offers various methods for managing user authentication, retrieving user information, and executing transactions on the blockchain. With the NEXT SDK, your dApps can integrate with the Bitkub NEXT Wallet Authentication services and submit transactions directly on the network.

Prerequisites

Before you install or interact with the NEXT SDK, we highly recommend generating the necessary keys, including the **Client ID** and **Project ID**. For the KUB Testnet environment, please visit the [KUB Playground](#). For the KUB Mainnet environment, please send an email to contact@kubchain.com.

Installation

To install the NEXT SDK, you can use either **npm**, **pnpm**, or **yarn**.

Shell

```
npm install @bitkub-chain/sdk.js
```

Shell

```
pnpm install @bitkub-chain/sdk.js
```

Shell

```
yarn add @bitkub-chain/sdk.js
```

Initialization

To initialize the SDK, create **lib/bitkubchain-sdk/index.ts** at the root project with the `clientId` and `projectId`.

JavaScript

```
import { initializeSDK, Network } from '@bitkub-chain/sdk.js';

const clientId = 'your-client-id';
const projectId = 'your-project-id';
const network = Network.BKC_TESTNET;
const initOpts = {
  loginRedirectPath: '/oauth/callback',
};

const sdk = initializeSDK(clientId, projectId, network, initOpts);
```

IMPORTANT: Please declare your network as **Network.BKC_TESTNET** for KUB Testnet and **Network.BKC_MAINNET** for KUB Mainnet.

Authentication

To begin with authentication, you have to set up OAuth to allow your dApp to log in with Bitkub NEXT. The first step is to create **/oauth/callback** page, you can use the following code:

JavaScript

```
import { useEffect } from 'react'
import { sdk } from '@lib/bitkubchain-sdk'

export default function Page() {
  useEffect(() => {
    const code = new URLSearchParams(window.location.search).get('code')
    if (code) {
      exchange(code)
    }
  }, [])

  const exchange = async (code: string) => {
    await sdk.exchangeAuthorizationCode(code)
    window.close()
  }

  return (
    <div>
      <h2>Callback Page</h2>
      <p>Exchanging code for tokens...</p>
    </div>
  )
}
```

Redirect URI

You can visit the KUB Playground to configure your redirect URI for your dApp.

If you have not specified a **loginRedirectPath**, the default redirect URI will be **/oauth/callback**.

You should configure it as follows:

`http://your-application-domain.com/oauth/callback`

If you have specified a custom **loginRedirectPath**, update your redirect UI as follows:

`http://your-application-domain.com/your-custom-loginRedirectPath`

Login

Opens a login window for Bitkub NEXT.

JavaScript

```
await sdk.loginWithBitkubNext();
```

Logout

Logs out the current user.

JavaScript

```
await sdk.logout();
```

Exchange Authorization Code

Exchanges the OAuth code for access and refresh tokens.

JavaScript

```
const code = 'authorizationCode';  
await sdk.exchangeAuthorizationCode(code);
```

User Information

Get User Information

Returns an object containing user details.

```
JavaScript  
const userInfo = await sdk.getUserInfo();
```

Get Wallet Address

Fetches the user's wallet address.

```
JavaScript  
const walletAddress = await sdk.getUserWalletAddress();
```

Get Phone Number

Fetches the user's phone number.

```
JavaScript  
const tel = await sdk.getUserTel();
```

Get Email

Fetches the user's email address.

```
JavaScript  
const email = await sdk.getUserEmail();
```

Get User ID

Fetches the user's unique ID.

```
JavaScript  
const userID = await sdk.getUserID();
```

Balances

Native Balance

Returns the user's native balance in wei.

```
JavaScript  
const balance = await sdk.getBalanceNative();
```

NOTE: If the network is **Network.BKC_MAINNET** or **Network.BKC_TESTNET**, KKUB balance will be returned. Otherwise, the native currency of the network will be returned.

ERC-20 Token Balance

Returns the balance of a specific ERC-20 token in wei.

```
JavaScript  
const balance = await sdk.getBalance20(tokenAddress);
```

ERC-721 Token Balance

Returns the number of ERC-721 tokens owned by the user.

```
JavaScript  
const balance = await sdk.getBalance721(tokenAddress);
```

ERC-1155 Token Balance

Returns the quantity of a specific ERC-1155 token owned by the user.

```
JavaScript  
const balance = await sdk.getBalance1155(tokenAddress, tokenID);
```

Token Approval and Transfers

Users must approve an allowance before transferring KAP-20 token to ensure that only authorized smart contracts can spend their tokens, preventing unauthorized access and protecting their assets.

Get Token Allowance

Gets the current token allowance for a given spender in wei.

```
JavaScript
const allowanceAmount = await sdk.getAllowanceToken(tokenAddress,
spenderAddress);
```

Approve Token

Approves a spender to use a specified amount of tokens.

```
JavaScript
const result = await sdk.approveToken(tokenAddress, amount, spenderAddress);
```

Transfer Native Token

Transfers native KUB to a recipient.

```
JavaScript
const result = await sdk.transferNative(toAddress, amount);
```

Transfer ERC-20 Token

Transfers a specified ERC-20 token to a recipient.

```
JavaScript
const result = await sdk.transfer20(tokenAddress, toAddress, amount);
```

NFT Approval and Transfers

Users must approve before transferring NFTs to ensure that only authorized smart contracts can send their NFTs, preventing unauthorized access and protecting their assets.

NFT Approval

Checks if a given operator is approved for the user's NFT.

```
JavaScript
const isApprovedNFT = await sdk.getIsApprovedNFT(tokenAddress,
operatorAddress);
```

Approve NFT

Grants approval to an operator to manage the user's NFT.

```
JavaScript
const result = await sdk.approveNFT(tokenAddress, operatorAddress);
```

Transfer ERC-721 Token

Transfers an ERC-721 token to a recipient.

```
JavaScript
const result = await sdk.transfer721(tokenAddress, toAddress, tokenID);
```

Transfer ERC-1155 Token

Transfers a quantity of an ERC-1155 token to a recipient.

```
JavaScript
const result = await sdk.transfer1155(tokenAddress, toAddress, tokenID,
amount);
```


Custom Transaction

Sends a custom smart contract transaction using human-readable ABI and parameters.

```
JavaScript
const result = await sdk.sendCustomTx(toAddress, functionReadableABI,
methodParams);
```

Fetch Transaction Details

Returns the details of a submitted transaction using its queue ID.

```
JavaScript
const details = await sdk.getTransactionDetails(queueId);
```

Smart Contract Design for NEXT SDK

To allow Bitkub NEXT users to write data on a smart contract, you need to design the smart contract with additional functions. The additional functions are called through the **CallHelper** smart contract to execute commands on behalf of the Bitkub NEXT users. The function must also use an extra **address _bitkubNext** parameter at the end of the function.

Example

This function for buying an NFT requires 2 parameters, which are **_recipeIndex** and **_amount**. MetaMask or other wallets can call this function to buy an NFT.

```
JavaScript
function buyNft(uint256 _recipeIndex, uint256 _amount) external
```

The function must use an extra **address _bitkubNext** parameter at the end of the function to allow Bitkub NEXT users to call this function to buy an NFT.

```
JavaScript
function buyNft(uint256 _recipeIndex, uint256 _amount, address _bitkubNext)
external onlySdkCallHelperRouter
```

You can now apply the human-readable ABI when you are calling the NEXT SDK for your custom transaction.

JavaScript

```
const functionReadableABI = "buyNft(uint256 _recipeIndex, uint256 _amount,  
address _bitkubNext)"  
const methodParams = [1, 1]
```

SDK CallHelper Router

When an end user invokes a function through the SDK library, the **msg.sender** interacting with your smart contract will be **sdkCallHelperRouter**. To ensure that only authorized calls are made to this function, you must include a modifier to check that **msg.sender** equals **sdkCallHelperRouter**.

Example Modifier

JavaScript

```
modifier onlySdkCallHelperRouter() {  
    require(msg.sender == sdkCallHelperRouter, "Authorization: restricted only  
SDK CallHelper Router");  
    _;  
}
```

IMPORTANT: Confirm that the address of the **sdkCallHelperRouter** for **Network.BKC_TESTNET** is **0x96f4C25E4fEB02c8BCbAdb80d0088E0112F728Bc**.

Appendix A

Introducing KUB Playground

KUB Playground is a platform designed for users to create a client ID and a project ID on KUB Testnet. This platform provides developers with the tools and resources needed to conduct comprehensive testing and experimentation with the latest NEXT SDK updates. Users can perform testing, validate integrations, and ensure optimal functionality in their decentralized applications.

Quick Start Guide on KUB Playground

Redirect to <https://playground.kubchain.com> to try our KUB Playground. The KUB Playground consists of 6 primary steps:

1. User Information
2. Create Client ID
3. Create Project ID
4. Transaction System
5. Queue System
6. Client Configuration

Get Started: Login with Bitkub NEXT

Begin by clicking on the **Login with Bitkub NEXT** button to access KUB Playground.



Step 1: User Information

This section will display all your information, such as your wallet address and phone number. KUB Playground will also display your KKUB balance.

NOTE: To use KUB Playground, you must have at least 5 KKUB in your Bitkub NEXT Wallet on Mainnet. You can get KUB on any listed CEXs and withdraw KUB to your Bitkub NEXT address. Your withdrawn KUB will be automatically wrapped into KKUB.



Step 2: Create Client ID

Enter your Project Name and Redirect URIs below. The Project Name identifies your project, while the Redirect URI is the URI which users will be redirected to your application after they grant access to their account.

The screenshot displays the 'Bitkub Chain Playground' interface. At the top, there is a 'TESTNET' label and a 'Disconnect' button next to the address '0xd7f5...fff1'. Below this, a vertical progress bar on the left indicates the current step: '2 Create Client Id'. The main form area contains a 'Project Name' field with the value 'NFT Marketplace' and a 'Redirect URIs' section with two input fields containing 'http://marketplace.nft/oauth/callback' and 'http://marketplace.nft/oauth/redirect', each with a close button (X). A '+' button is located below the URIs, and a 'Submit' button is at the bottom of the form. The progress bar also lists subsequent steps: '3 Create Project ID', '4 Create Transaction System (on-chain)', '5 Create Queue System', and '6 Done'.

Bitkub Chain Playground

0xd7f5...fff1 [Disconnect](#)

✓ User Information

2 Create Client Id

Project Name

NFT Marketplace

Redirect URIs

http://marketplace.nft/oauth/callback ×

http://marketplace.nft/oauth/redirect ×

+

Submit

3 Create Project ID

4 Create Transaction System (on-chain)

5 Create Queue System

6 Done

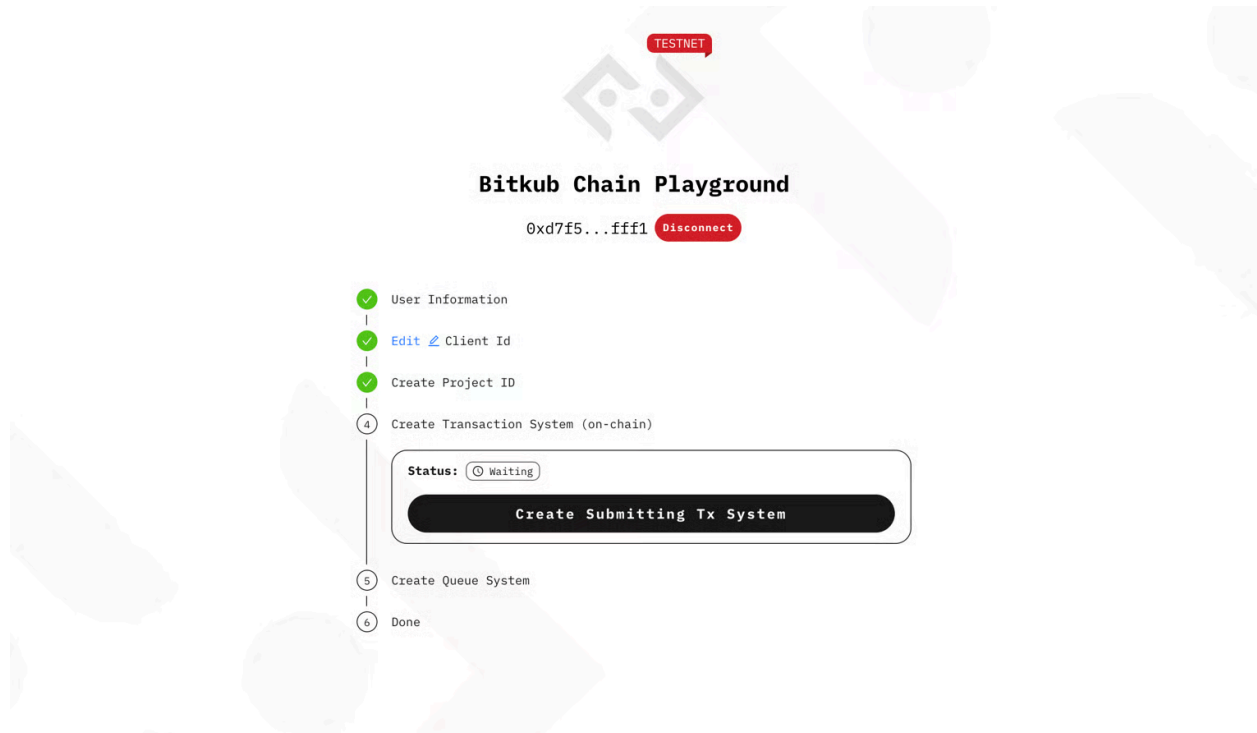
Step 3: Create Project ID

Click Create Project ID to generate your Project ID.



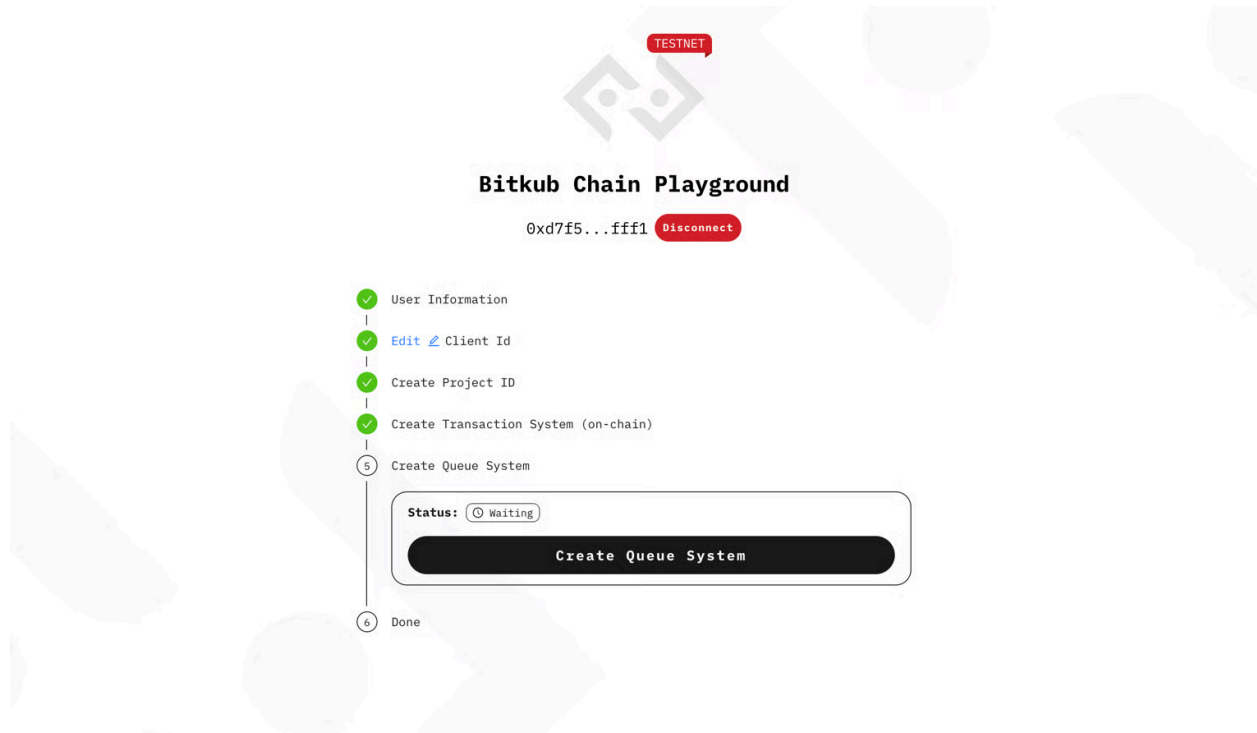
Step 4: Create Transaction System (On-chain)

Click on Create Submitting TX System to register for the approval and submission service and set up a transaction system for your project.



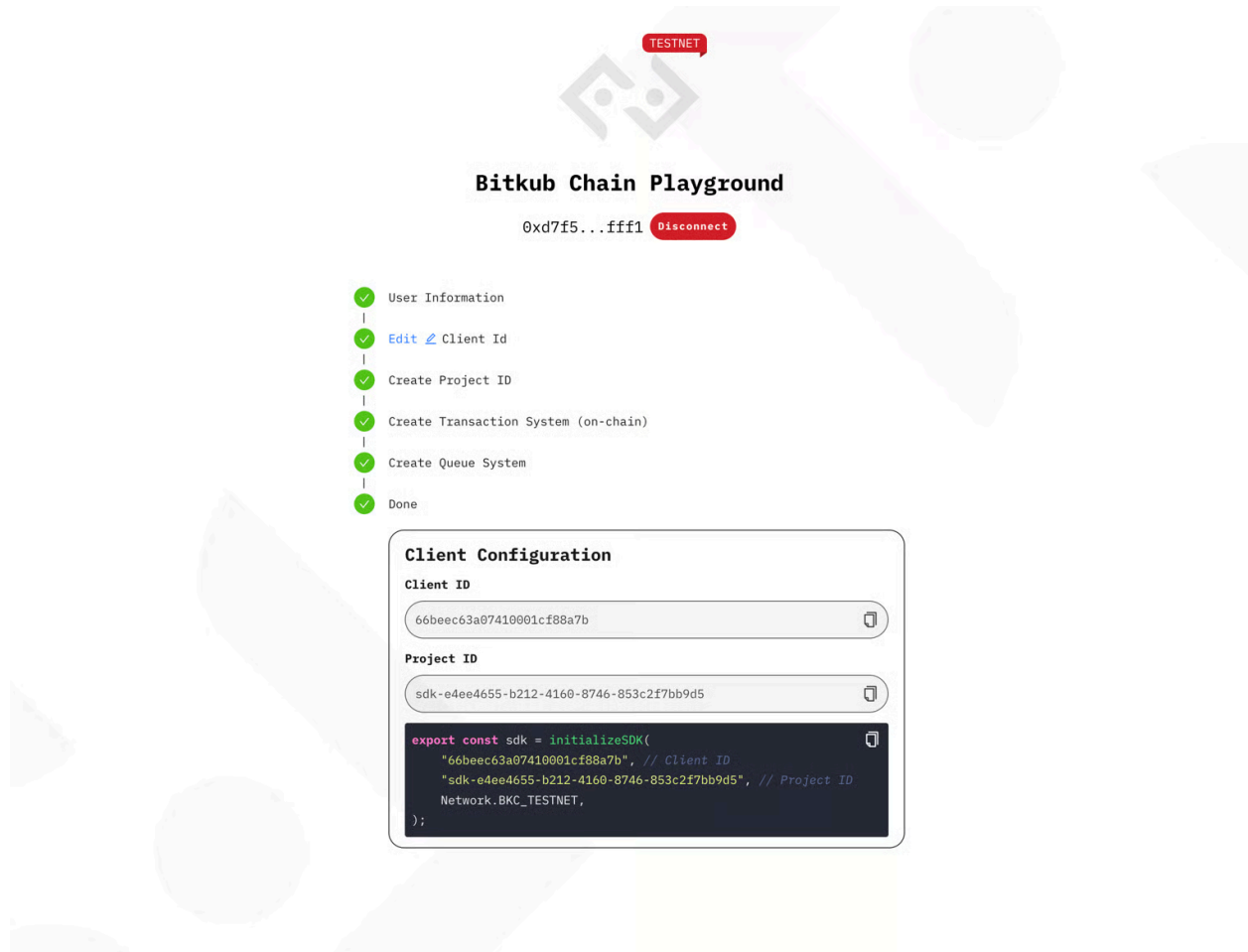
Step 5: Create a Queue System

Click on Create Queue System to create a queue system for sending your project's ABI to our server and SDK Services.



Step 6: Client Configuration

Once all steps are completed, your client configuration will be displayed. This includes your Client ID, Project ID, and sample code to initialize the SDK in your application.



Appendix B

Balances Properties

Property	Description	Type	Example
tokenAddress	The address of the token contract	string	0x67eBD850304c70d983B2d1b93ea79c7CD6c3F6b5
tokenId	The ID of the specific token or asset	BigNumberish	1

Transactions Properties

Property	Description	Type	Example
tokenAddress	The address of the token contract	string	0x67eBD850304c70d983B2d1b93ea79c7CD6c3F6b5
toAddress	The address of the recipient	string	0x1234567890abcdef1234567890abcdef12345678
amount	The amount of tokens to be transferred in wei	BigNumberish	1 Ether = 1000000000000000000
tokenId	The ID of the specific token or asset	BigNumberish	1
functionReadableABI	The human-readable format of the function ABI	string	buyNft(uint256 _recipientIndex, uint256 _amount, address _bitkubNext)
methodParams	The parameters used in the method call	Array	[1, 1]

SubmitTransactionResponse

Property	Type	Description
approvalExpireTime	number	The expiration time of the approval in Unix timestamp format.
approvalStatus	pending approved	The current status of the approval.
approvalUrl	string	The URL to approve the transaction.
queueId	string	The unique identifier for the transaction queue.

Token Approval Properties

Property	Description	Type	Example
tokenAddress	The address of the token contract	string	0x67eBD850304c70d983B2d1b93ea79c7CD6c3F6b5
amount	The allowance amount (in Wei) authorizes a smart contract to spend a specified number of your tokens on your behalf.	BigNumberish	1 Ether = 1000000000000000000
spenderAddress	(Optional) If transfer using transferNative or transfer20, ignore this property. The address of a smart contract or user that has been granted permission to spend tokens on behalf of another address.	string	0x123eCF850304c70d983B2d1b93ea79c7CD6c3H6b7

NFT Approval Properties

Property	Description	Type	Example
tokenAddress	The address of the token contract	string	0x67eBD850304c70d983B2d1b93ea79c7CD6c3F6b5
operatorAddress	(Optional) If transfer using transfer721 or transfer1155, ignore this property. The address of a smart contract or user that has been granted permission to send NFT on behalf of another address.	string	0x123eCF850304c70d983B2d1b93ea79c7CD6c3H6b7

SubmitTransactionResponse

Property	Type	Description
errorMessage	string	A message describing any error that occurred.
queueId	string	The unique identifier for the transaction queue.
signature	string	The signature associated with the transaction.
status	TransactionStatus	The current status of the transaction.
txHash	string	The transaction hash.
createdTime	number	The timestamp when the transaction was created.
updatedAt	number	The timestamp when the transaction was last updated.